

A3 · curl, nc, and ping

CS425 Computer Networks · in class activity · everything runs on onyx.boisestate.edu

TURN THIS IN ON PAPER

before you leave the room
graded pass/fail

Names _____ Date _____

Why you are doing this

In A1 you learned to read `dig`. Today, three more tools, and then you have everything A4 needs. `curl` talks HTTP and tells you where the time went. `nc` talks nothing, so you have to. `ping` is the odd one out, and by Round 3 you will be able to say why. Three rounds, one sheet, write down what you see.

ROUND 1 · curl: HTTP, AND WHERE THE TIME GOES

Lines starting with `>` are what you sent. Lines starting with `<` are what the server replied.

```
curl -v http://example.com/
```

FIND THIS	WRITE IT HERE
The request line you sent	
The <code>Host:</code> header	
The status line that came back	
The <code>Content-Type</code>	

COMMAND	WHAT IT CHANGED
<code>curl -I http://example.com/</code>	
<code>curl -IL http://www.boisestate.edu/</code>	
<code>curl -H 'User-Agent: cs425' -v http://example.com/</code>	
<code>curl --connect-timeout 3 -m 5 http://example.com/</code>	

Where the time actually goes · this is what is inside probe

```
curl -o /dev/null -s -w 'dns=%{time_namelookup} connect=%{time_connect} tls=%{time_appconnect} ttfb=%{time_starttransfer} total=%{time_total} code=%{http_code}\n' https://www.boisestate.edu/
```

DNS	CONNECT	TLS	TTFB	TOTAL	CODE

- Which phase took the longest?

- `dns` is name resolution and `connect` is the TCP handshake. What is happening between `connect` and `ttfb`?

- Run it against `http://` instead of `https://`. Which number collapses to zero, and what does that tell you about what it was measuring?

- Run `declare -f probe`. Which two of the five does your A2 function report?

ROUND 2 · nc: BE THE CLIENT YOURSELF

nc knows no protocol at all. It opens a connection and moves bytes, which means you get to be the protocol. That is exactly what P1 asks you to write.

```
nc -vz onyx.boisestate.edu 22    # something is listening
nc -vz onyx.boisestate.edu 9     # nothing is
```

PORT	WHAT NC PRINTED, WORD FOR WORD	HOW LONG
22		
9		

Those are two completely different failures. Telling them apart is most of A4.

Speak HTTP by hand

```
{ printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n'; sleep 3; } | nc example.com 80
```

Why the sleep 3? Onyx has nmap-ncat, which closes the connection the moment stdin runs out. Without the sleep, your request goes out and nc hangs up before the reply lands. Run it once without, add `-v`, and read the bytes sent versus bytes received line.

1. What was the very first line the server sent back?

2. Drop the **Host:** header and run it again. What changed, and why does HTTP/1.1 require it?

3. Change every `\r\n` to `\n`. What does the server do? **SMTP has the same rule**, and this was the most common bug in P1. You cannot see it in a terminal; you have to know it.

Listen, and be listened to

You are all on one machine, so one of you can be the server. Bind to **127.0.0.1**, never **0.0.0.0**, and kill it when you are done.

```
PORT=$((10000 + $(id -u) % 20000)); echo "$PORT"
ss -ltn | grep ":$PORT" || echo "free"
nc -l 127.0.0.1 "$PORT"           # blocks, waiting. Another member: nc 127.0.0.1 <PORT>
```

Whose port did you connect to, and what did you send? Confirm with `ss -ltn` that nothing of yours is still listening afterwards.

ROUND 3 · ping: THE ONE THAT IS NOT AN APPLICATION TOOL

Everything above spoke an application protocol. This one does not: ICMP rides directly in IP, and has no ports and no idea what a service is.

```
ping -c 5 onyx.boisestate.edu      # also try: -i 0.5  -s 1000  -W 2  -w 5  -n  -q
ping -c 3 example.com             # the host you fetched a page from in Round 1
```

TARGET	PACKET LOSS	DID CURL REACH IT EARLIER?
onyx.boisestate.edu		
example.com		

Those two rows contradict each other. That contradiction is the whole point of this round, and of Round 3 in A4.

1. What layer does ICMP live at, and what does that mean about ports?

2. Onyx answers ping. Does that prove the SSH server on it is running? Why not?

3. **example.com** does not answer ping, and also served you a web page. So what did the failed ping actually tell you? Name at least two different things that could produce that result.

If you finish early: write `probes` in `~/ .bashrc`, a second function that prints the full five phase breakdown from Round 1. Then run `probe onyx.boisestate.edu 9` and compare it to what `nc -vZ` said about the same port in Round 2:

Graded pass/fail on a serious attempt at every round. Hand this in before you leave, with every name at the top.